Programmable Timers

ECE 2564, Embedded Systems

Fall 2025

- How to measure elapsed time
- Import **this repository**

Motivation

- If you write programs for a general-purpose computer often you are interested only in performance
  - Complete the job as fast as possible
- When you program microcontrollers it can be very important to do the right things AT THE RIGHT TIMES
  - Not as fast as possible

- Examples:
  - Toggle an LED every half-second
  - Detect two button presses and count the number of milliseconds between them
  - Fire an engine's spark plug after a particular delay

Real-time Computing

- Almost all embedded systems perform some type of real-time computing
- And so time needs to be measured
- Based on time we can often infer distance
- Events and deadlines may be of interest

- More examples: GPS localization, touch-screen control, antilock brakes

- To measure time many systems rely on programmable (hardware) timers

Illustrative Example – Pseudocode

Consider a program that must perform two tasks:

```
int main()
{
   initialize the system
   while(1)
   {
      if BUTTON1 is pressed
         turn LED1 on
      else
         turn LED1 off

      toggle LED2 at every 500 ms
   }
}
```

Illustrative Example – Pseudocode

```
int main()
{
   initialize the system
   while(1)
   {
      if BUTTON1 is pressed              // We know how
         turn LED1 on
      else
         turn LED1 off                   // to do this

      toggle LED2 at every 500 ms        // How to do this?
   }
}
```

Illustrative Example – Pseudocode

```
int main()
{
   initialize the system
   while(1)
   {
      if BUTTON1 is pressed
         turn LED1 on
      else
         turn LED1 off
   // What about this? Pros and cons?
      for(int i=0; i<BIGVALUE; i++)
         do nothing for 500ms
      toggle LED2
   }
}
```

- Problem #1: this implementation uses a *blocking* delay, which generally is a bad choice
- Problem #2: what is the value of BIGVALUE that makes this work?

What is a Good Alternative?

- Interface a *hardware counter* to the CPU
  - Design the hardware so that the counter is updated at known, regular intervals
  - Provide a mechanism for software to read the counter's contents
  - Provide a mechanism for software to control the counter
    - initialize, start, stop
- This unit is sometimes referred to as a *timer module* or *timer subsystem*

Illustrative Example (continued)

```
// During initialization set up the timer so that
// counting down from the initial value to 0
// corresponds closely to 500 ms
while(1) {
    if BUTTON1 is pressed
        turn LED1 on
    else
        turn LED1 off

    if(timerExpired()) {  //Nonblocking !!
        Toggle LED2
        Re-start timer
        }
}
```

A Timer-Based Solution

```
int main(void)
{
   initialize();
   while (1) {
      if (!S1isPressed())
         TurnOff_Launchpad_LED1();
      else
         TurnOn_Launchpad_LED1();
      if (Timer32_getValue(TIMER32_0_BASE) == 0)
      {
         // Toggle the LED
         Toggle_Launchpad_Green_LED2();
         // since the timer is in one-shot mode,
         // we have to restart it to get a periodic behavior
         Timer32_setCount(TIMER32_0_BASE, ONE_SEC_COUNT);
         Timer32_startTimer(TIMER32_0_BASE, true);
      }
   }
}
```

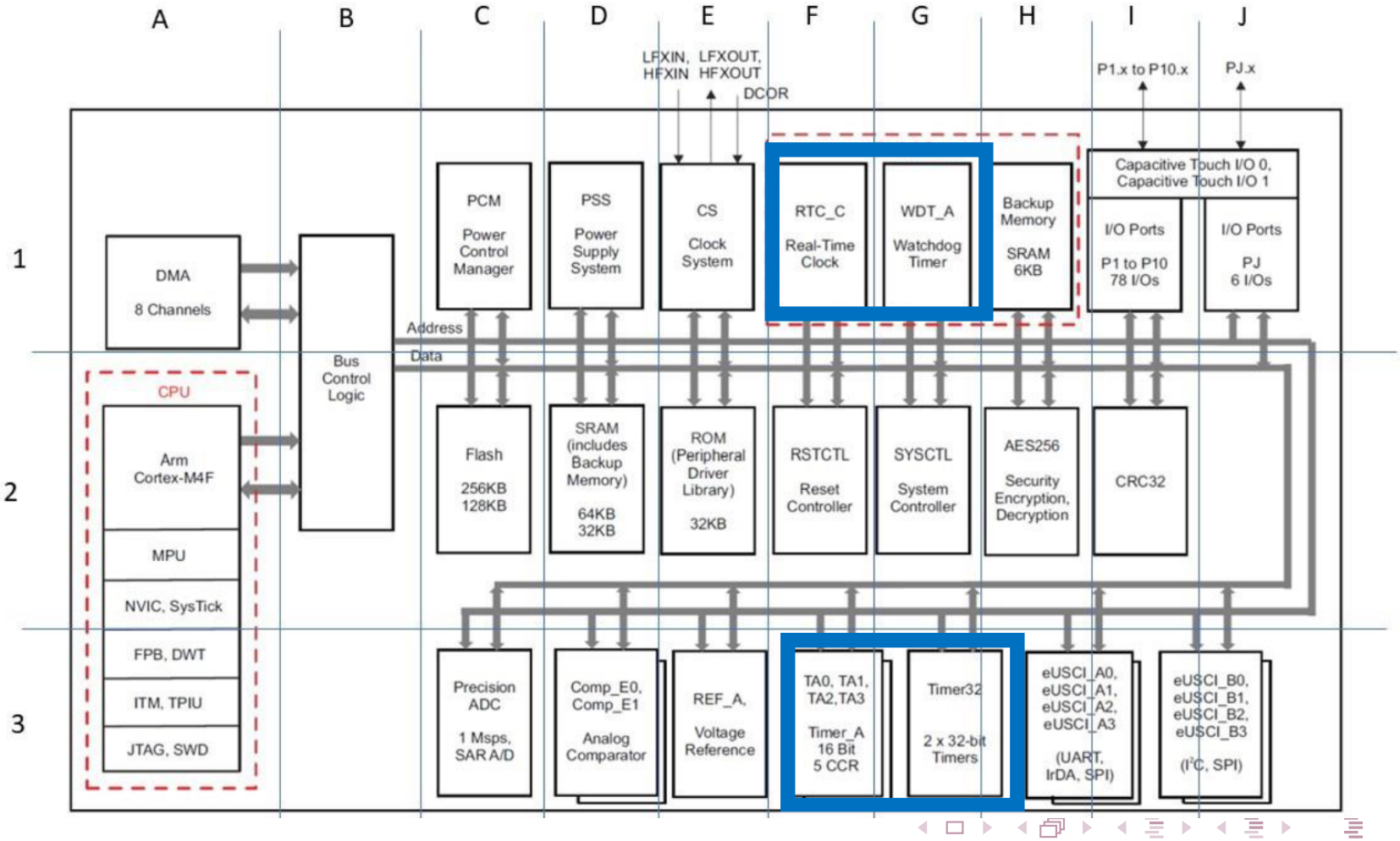
Hardware Timer vs. Software Timer

You can compare a hardware timer with a software delay loop

```
#ifdef Timer32
   // If the timer32 has expired
   if (Timer32_getValue(TIMER32_0_Base) == 0)
   {
      // Toggle the LED
      Toggle_Launchpad_Green_LED2();

      // Since the timer is in one-shot mode, it must be restarted
      // to get a periodic behavior
      Timer32_setCount(TIMER32_0_BASE, ONE_SEC_COUNT);
      Timer32_startTimer(TIMER32_0_BASE, true);
   }
#else
   int i;
   // an approximate 1 second delay with simple loop
   for (i=0; i<240000; i++);
   Toggle_Launchpad_Green_LED2();
#endif
```

MSP432 Block Diagram -- Timer

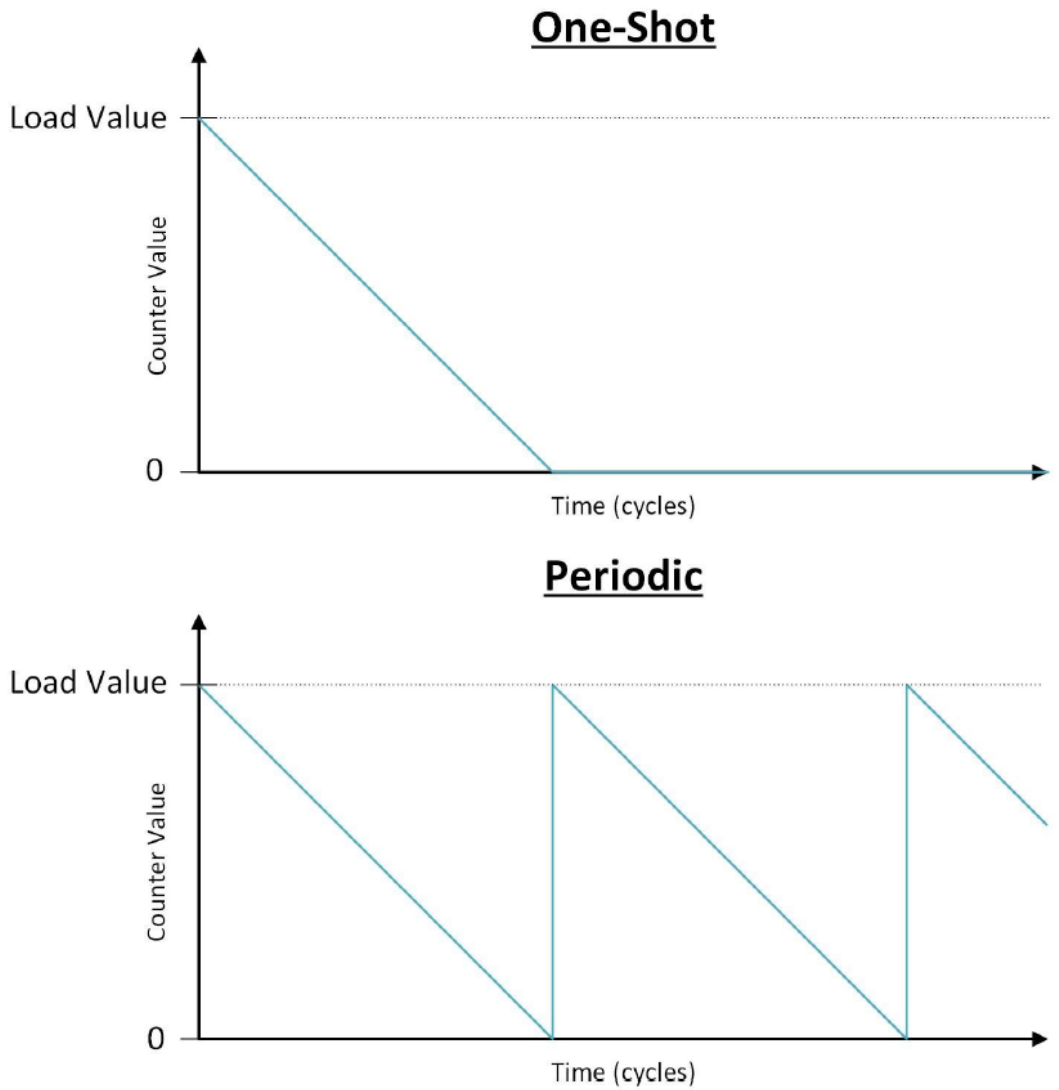


The MSP432 Timer32 module

There are two independent, programmable counters

- 32-bit down-counters
- Each can be used in 16-bit mode
- Referenced in the DriverLib using
  - TIMER32_0_BASE and TIMER32_1_BASE
- There are three modes of operation
  - *Free-running mode:* Count down from `0xff...ff`, and repeat when 0 is reached
  - *Periodic timer mode:* Countdown from value in `Load Register` and repeat when 0 is reached
  - *One-shot timer mode:* Count down to 0 and then stop counting

Timer Modes


One-Shot
Load Value
Counter Value
0
Time (cycles)
Periodic
Load Value
Counter Value
0
Time (cycles)

Timer Logic (One-Shot Mode)

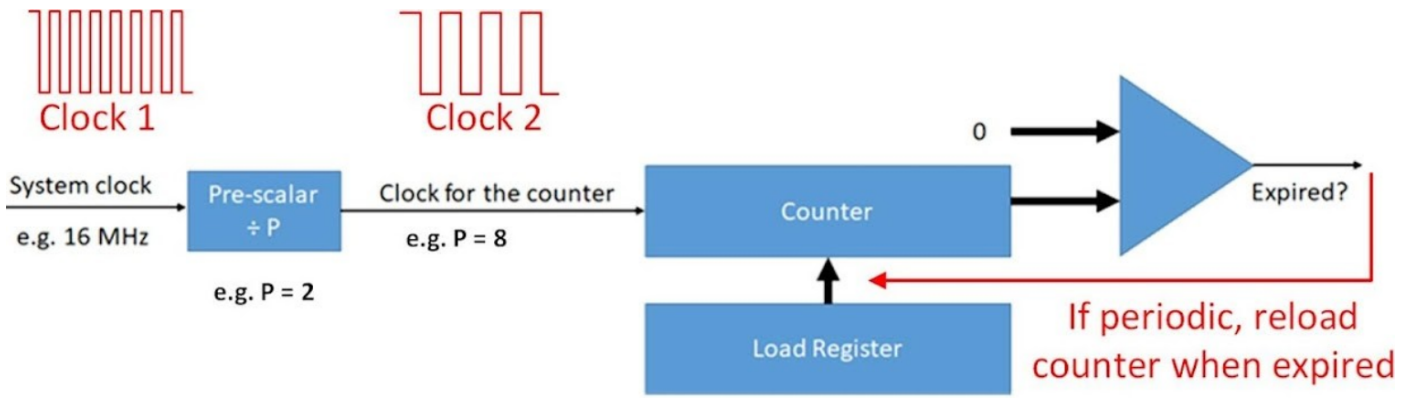
1. Load a value into a counter
2. Counter decrements based on a counter-clock
3. Compare counter with 0
   - if counter $==$ 0, signal expire $=$ true

For an n-bit counter, the largest unsigned value we can put in the counter is

$$2^n - 1$$

Pre-Scalar (assume $n = 32$)

- What if we want the timer to start with a larger value than $2^{32} - 1$?
- A *pre-scalar* is used to make the counter count slower by dividing the clock frequency by P
  - The counter clock is divided by P in hardware, so the counter decrements at a slower rate
  - Small values of P, such as 1 and 2, are used for accurate time measurements but for shorter intervals
  - Larger values of P, such as 8 and 16, are used for less accurate time measurements but longer intervals



One-Shot Timer Formula

- N: counter load value
- Countdown sequence: N, $N-1$, $N-2$, $\ldots$, 1, 0
- f_S = system clock frequency
- T_S = system clock period $= 1/f_S$,
- P = pre-scalar factor
- T_C = counter clock period
- $T_C = \mathrm{P} \times T_s$
- Expiration Time $= N \times T_C = N \times \mathrm{P} \times T_s =$ one-shot-timer interval

- Example: $N = 100$, $\mathrm{P} = 10$, $f = 10\,\mathrm{MHz}$
- One-shot-timer interval $= 100 \times 10 * (10\,\mathrm{MHz})^{-1} = 100\,\mu\mathrm{s}$

Programming the Timer

- Initializing the timer and counter load value and starting the timer for one-shot mode

```
Timer32_initModule(TIMER32_0_BASE, // There are two timers,
       // we are using the one with the index 0
    TIMER32_PRE-SCALAR_1, // The pre-scalar value is 1;
       // the clock is not divided before feeding the counter
    TIMER32_32BIT, // The counter is used in 32-bit mode;
       // the alternative is 16-bit mode
    TIMER32_PERIODIC_MODE); // This option is irrelevant
       // for a one-shot timer

Timer32_setCount(TIMER32_0_BASE, ONE_SEC_COUNT);  // The count
       //related to the period of the timer
Timer32_startTimer(TIMER32_0_BASE, true); // second parameter is
       // the one-shot boolean -- true: start the timer
       // in one-shot mode. This means the timer stops when it reaches 0
```

Check our Understanding

NOTE: For Timer32, $1 <= \text{P} <= 256$, where the pre-scalar, P, is an integer

What are the longest and shortest intervals, in seconds, that Timer32 can measure when the system clock frequency, f_S, is 48 MHz?

Recall: One-shot-timer interval $= N \times \text{P} \times T_S$

Check our Understanding

Consider a Timer32 timer where the system clock is 48 MHz, the pre-scalar is 256 and the counter load value is 999. The timer is started.

What is the value of the timer after 528 µs?

Summary

- Motivation: The importance of measuring time
  - Time itself has to be measured
  - The time measurement may be related to a distance calculation
  - A time measurement is needed for controlling a system response
- Real-time computing versus general-purpose computing
- Hardware timers introduced